PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Contents

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: Context-Based Applicant Tracking System **(CATS)**

Version: 1.0

1. Project Overview

- **Problem Statement:** Resumes are inherently biased and easily exaggerated, rendering traditional keyword-matching ATS completely blind to a candidate's actual capability. To filter out fake or overhyped candidates, HR personnel are forced to manually play "digital detective" cross-checking LinkedIn profiles, scrolling through GitHub repositories, and Googling portfolios for every single applicant. This manual verification process creates a massive bottleneck, turning a scalable hiring process into a tedious, unscalable nightmare that leads to recruiter burnout and costly bad hires.

- **Target Domain:** Deep-Talent Verification, Intelligent Recruitment Automation, and HR Tech.

- **Proposed Solution Summary:** Our Context-Based ATS eliminates the guessing game by shifting the recruiter's role from a "manual background checker" to a "decision maker." Instead of just reading resumes, the system acts as an automated investigative pipeline: It extracts resume data, deploys targeted scrapers to fetch real-world digital footprints (GitHub commits, LinkedIn tenure), and merges this evidence into a unified JSON format. An LLM then evaluates this enriched context against the job description, delivering deeply verified candidate rankings and automated, highly specific interview questions reducing days of manual stalking to mere seconds.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Background & Business Objective

- **Background of the Problem:** The modern hiring pipeline is fundamentally broken by a single point of failure: Trust. A candidate claims to be an "Expert in Python," but traditional ATS simply believes it if the keyword is present. To uncover the truth, a recruiter must manually open multiple tabs, verify the claim against GitHub, check LinkedIn for actual job titles, and piece together the context themselves. At scale, when a single job post attracts 500 applicants, this manual verification is mathematically impossible, forcing companies to either blindly trust resumes or arbitrarily reject good candidates.
- **Importance of Solving This Issue:** By automating the evidence-gathering phase, we introduce "Proof-of-Work" into recruitment. A candidate with a plain resume but a stellar GitHub contribution graph will automatically out-rank a candidate with a beautifully designed but empty portfolio. This completely neutralizes resume-padding, drastically reduces time-to-hire, and ensures that only verified, contextually relevant talents reach the interview stage.
- **Strategic Fit / Impact:** This moves beyond standard NLP text extraction into the realm of Agentic AI workflows. It demonstrates how LLMs can be orchestrated with web scraping tools to perform complex, multi-step reasoning aligning perfectly with the future of AI-native enterprise automation.

3. Product Purpose

- **Automate Skill Verification with Minimal Human Interaction:** The system is designed to reduce human involvement as much as possible by automatically collecting and validating candidate data. Recruiters no longer need to manually check multiple platforms, as the system handles verification in the background.

© UMHackathon 2026

- **Establish "Proof-of-Work" for Candidates :** Instead of trusting resume claims, the product analyzes real contributions from platforms like GitHub and LinkedIn. This ensures that candidates are evaluated based on actual work and performance rather than self-declared skills.
- **Improve Accuracy and Fairness in Hiring Decisions** : By relying on verified data instead of keyword-based resumes, the system reduces bias and prevents resume padding. Candidates with genuine skills are more likely to be recognized, even if their resumes are simple.
- **Reduce Time-to-Hire** : Automated evidence gathering and evaluation significantly speed up the screening process. Recruiters can focus only on high quality, pre-verified candidates, making the hiring pipeline more efficient.
- **Enable AI-Driven Decision Making** : The product uses advanced AI workflows to analyze, compare, and rank candidates based on relevance and authenticity. This allows for smarter and more consistent hiring decisions at scale.
- **Rebuild Trust in the Recruitment Pipeline** : By shifting from assumption-based screening to evidence based evaluation, the system restores trust in hiring. Companies can confidently short-list candidates knowing their skills have been validated.

3.1. Main Goal of the System

- **To automate and validate candidate skill verification using real-world evidence with minimal human interaction :** The system's core purpose is to replace manual resume screening with an AI-driven process that gathers, analyzes, and verifies candidate data from external sources. By doing so, it ensures that hiring decisions are based on proven skills ("Proof-of-Work") rather than self-reported claims, while significantly reducing recruiter effort and improving efficiency, accuracy, and trust in the hiring process.

© UMHackathon 2026

3.2. Intended Users (Target Audience):

- Technical Recruiters & Talent Acquisition Specialists
- Engineering Managers & Tech Leads
- Startup Founders & Hiring Managers

4. System Functionalities

4.1. Description: The system operates as a continuous, multi-stage intelligence pipeline. Raw applicant data enters the Resume Portal and is structurally broken down (Extractor). These specific data points trigger targeted Scrapers to gather live, unstructured web data. Both datasets are synthesized into a clean JSON payload, which is then fed into the LLM for contextual evaluation, semantic ranking, and strategic question generation, finally surfacing on the Employer Portal.

4.2. Key Functionalities:

- **Deep-Scan Extraction:** Ingests unstructured PDFs/DOCX and parses them into a highly structured JSON schema (skills, tenure, URLs).
- **Digital Footprint Scraping:** Actively queries public APIs/web pages (LinkedIn for role verification, GitHub for code frequency/languages, Google for portfolio presence) based on extracted identifiers.
- **Contextual Evidence Matching:** The LLM cross-references the "Claimed Skills" (from resume) against "Proven Skills" (from scraper output) to generate a Verification Score.
- **Aggressive Interrogation Engine:** Automatically generates bespoke interview questions designed specifically to test the "gaps" or unverified claims found during the LLM evaluation phase.

© UMHackathon 2026

4.3. AI Model & Prompt Design

4.3.1. Model Selection: Z.AI GLM is chosen for its superior JSON-structuring capabilities and complex multi-step reasoning. The model must be able to hold the "memory" of the job description while simultaneously analyzing two entirely different data contexts (resume vs. scraped web data).

4.3.2. Prompting Strategy: Multi-step Agentic Prompting. Step 1: Instruct the LLM to parse the extracted data into JSON. Step 2: Prompt the LLM to compare the resume JSON against the job description and requirements. Step 3: Force the LLM to output a strict ranking rationale based *only* on provided evidence.

4.3.3. Context & Input Handling: Scraped web data (like a massive GitHub README) can exceed token limits. The system implements a "Relevance Chunking" mechanism, which utilizes separate scripts to scrape from the resume and web portfolios and only passing relevant data to the LLM. This saves the LLM the work of analyzing and scraping the content itself.

4.3.4. Fallback & Failure Behavior: If a candidate has no online footprint (or Scrapers are blocked), the system gracefully defaults to standard Resume-to-JD semantic matching, but attaches a prominent "Unverified Profile" flag to the Employer Portal to inform the recruiter of the lower confidence level.

5. User Stories & Use Cases

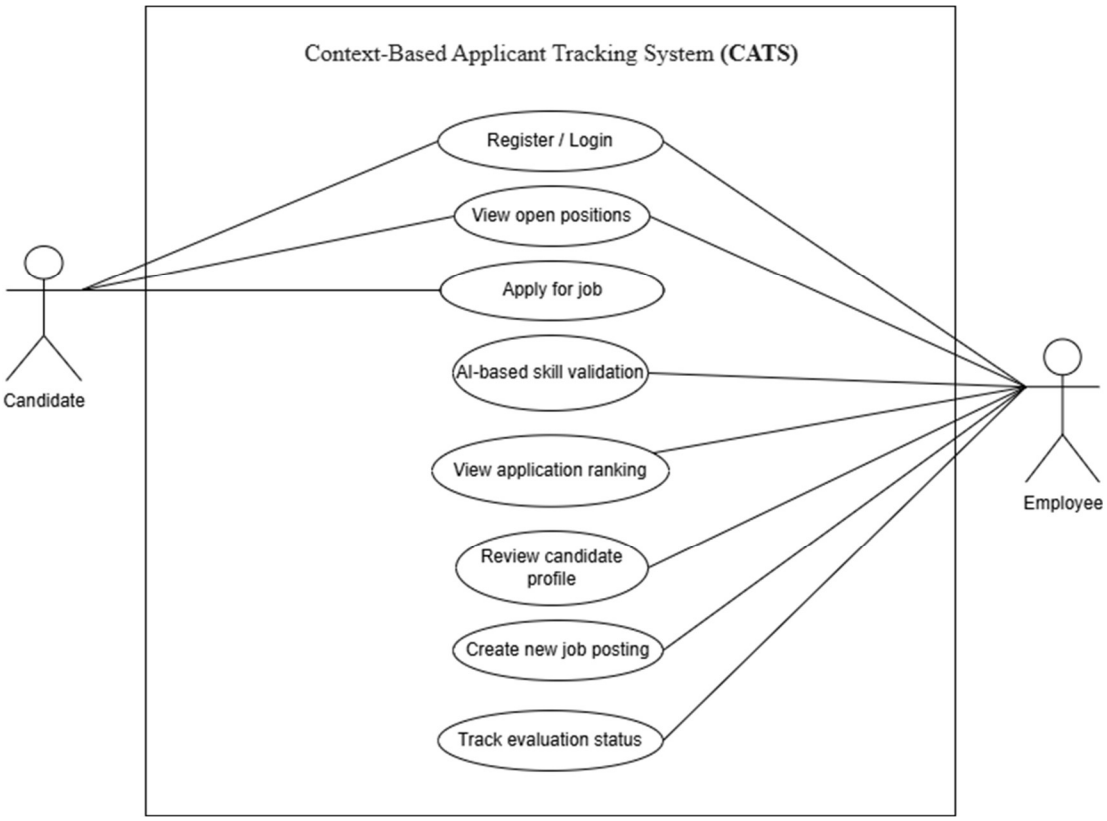
- **User Stories:** “As a Technical Recruiter, I want to upload a Job Description and a batch of resumes, so that the system automatically ranks candidates based on verified proof-of-work rather than just keywords.”
- "As a Hiring Manager, I want to see a side-by-side comparison of a candidate's 'Claimed Skills' vs. 'Proven Skills' (from GitHub/LinkedIn), so that I can immediately identify resume padding."

© UMHackathon 2026

- **Use Cases (Main Interactions):**

  Evidence Gathering Flow: User uploads Resume PDF → System extracts JSON data (skills, URLs) → Scraper module triggers to fetch live GitHub commit history and LinkedIn tenure → Data is merged into a unified context payload.

- **Use Case Diagram**.



© UMHackathon 2026

Email: umhackathon@um.edu.my

6. Features Included (Scope Definition)

- Dual-portal architecture (Applicant Resume Portal & Employer Dashboard).
- Intelligent Resume-to-JSON parser.
- Automated targeted scraping module (GitHub & LinkedIn public data integration).
- Evidence-based LLM ranking algorithm (Resumes are ranked by *proof*, not just keywords).
- Auto-generation of tailored interview questions based on data discrepancies.

7. Features Not Included (Scope Control)

- **Private Account Bypassing:** The system will only analyze publicly available data. It will not attempt to bypass LinkedIn privacy settings, log into private repositories, or use unauthorized credentials.
- **Automated Candidate Outreach:** The system will not send automated rejection or acceptance emails to candidates. It serves purely as an intelligence and ranking tool for the recruiter.
- **Enterprise HRIS Integration:** For the scope of this hackathon, the system will not integrate with existing enterprise HR software (e.g., Workday, BambooHR) via complex SSO or API webhooks.
- **Video/Audio Interviewing:** The system generates text-based interview questions but does not include a built-in video conferencing or voice-analysis module.

8. Assumptions & Constraints

- **LLM Cost Constraint:** Estimation is roughly 5,000 - 10,000 tokens per applicant. To keep inference costs manageable, the system implements "Relevance Chunking" (as stated in 4.3.3) to discard irrelevant text before sending it to the LLM, saving up a significant amount of token usage per query.
- **Technical Constraints (Scraping Limits):** GitHub API has a rate limit of 60 requests/hour for unauthenticated IPs, and LinkedIn aggressively blocks automated scrapers. The system assumes the use of basic web-scraping techniques which may occasionally fail or return CAPTCHAs during mass processing.

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Performance Constraints:** Because the system relies on live web scraping + LLM inference, processing 100 resumes will take significantly longer than a traditional keyword-matching ATS (estimated 15-30 seconds per resume). It is an asynchronous batch process, not real-time.
- **User Input:** The system assumes resumes are provided in standard digital formats (PDF) with selectable text. It cannot process scanned image resumes without an OCR pre-processing step (which is out of scope for V1).

9. Risks & Questions Throughout Development

- **Scraping Reliability & Blocking:** How do we ensure the system doesn't crash if LinkedIn changes its DOM structure or blocks our IP address mid-process? (Mitigation: Implement robust try-catch blocks and the "Unverified Profile" fallback).
- **LLM Hallucination in Verification:** What happens if the LLM falsely accuses a candidate of "resume padding" because it misinterpreted a scraped GitHub repository? (Mitigation: Prompt engineering must enforce "Benefit of the Doubt" logic, and UI must clearly show the raw scraped evidence to the recruiter for final human judgment).
- **Data Privacy & Compliance:** Are we legally allowed to scrape and store public LinkedIn/GitHub data for recruitment purposes? How long do we cache this data? (Note: Must ensure we only scrape public profiles and do not store PII like personal phone numbers unnecessarily).
- **False Positives in "Proof-of-Work":** A candidate might have 1,000 GitHub commits, but they are all minor (fixing typos in open source docs). How does the LLM distinguish between quantity of commits and quality of code?

© UMHackathon 2026
Email: umhackathon@um.edu.my